

Model based testing in incremental system development [☆]

Alexander Pretschner ^{*}, Heiko Lötzbeyer, Jan Philipps

Institut für Informatik, Technische Universität München, Boltzmannstrasse 3, Garching 85748, Germany

Received 31 December 2002; accepted 11 January 2003

Abstract

The spiraling nature of evolutionary software development processes produces executable parts of the system at the end of each loop. It is argued that these parts should consist not only of programming language code, but of executable graphical system models. As a main benefit of the use of more abstract, yet formal, modeling languages, a method for model based test sequence generation for reactive systems on the grounds of Constraint Logic Programming as well as its implementation in the CASE tool AutoFocus is presented.

© 2003 Elsevier Inc. All rights reserved.

Keywords: Cleanroom SW engineering; Constraint logic programming; Extreme programming; Incremental development; Rapid prototyping; Reactive systems; Test case generation

1. Introduction

With the recent discussion on what has become known as Extreme Programming (XP; Beck, 1999), evolutionary development processes are gaining popularity. The main advantage of evolutionary processes is that each development cycle leads to executable code: This satisfies programmers who get instant feedback on their activities (Brooks, 1986); it also satisfies customers who can watch the results of the development process, and who can influence development from the beginning to the end.

XP is based on common—usually object-oriented—programming languages, which is in a way an advantage: XP is rather lightweight, as it makes little demands on the infrastructure (except for compiler turn-around time). Conversely, however, XP fails to leverage recent advances in CASE tools, graphical description techniques, or algorithmic approaches to analysis and validation. XP is based on a rigorous testing discipline. For each increment, first test cases for the new functionality

must be specified; the test cases also serve as regression tests between increments.

In this paper, it is argued that incremental development processes for reactive systems should be based on higher-level description techniques and advanced test generation approaches. Reactive systems can, in principle, directly be implemented in programming languages like C or Ada. The benefit of using languages that are more abstract and closer to the application domain is discussed. These languages are called *modeling languages*, and artifacts written in these languages (behavior) *models*.

Because of the importance of test cases as functional specifications, as a debugging aid and as a basis for regression tests after design increments, it is important to support test case development in all development phases. This focus on testing is in contrast to other work on incremental modeling (Boger et al., 2000). More recent work (Ambler, 2002) advocates the use of models for developer tests. It leads to emphasize the use of a modeling languages with a clear semantics as a prerequisite for model based test generation.

Paper outline. After a general discussion of incremental design processes in Section 2, the modeling languages of the CASE tool AutoFocus (Section 3) using a simple traffic light example are presented. Section 4 is the core of this paper; it describes a technique for the

[☆] Supported by the DLR (project MOBASIS) and by the DFG under reference numbers Be 1055/7-2 (project KONDISK/IMMA) and Br 887/14-1 (project INKREA).

^{*} Corresponding author.

E-mail address: pretchn@in.tum.de (A. Pretschner).

automatic derivation of test cases on the grounds of Constraint Logic Programming (CLP). Section 5 compares the presented approach to the state of the art in related work. Section 6 concludes and gives an outlook on further work. This paper is an extended version of a workshop contribution (Pretschner et al., 2001a).

2. Incremental development

This section discusses incremental software development processes and argues for the use of modeling languages instead of programming languages. The role of testing in incremental development is clarified.

2.1. Development processes

One of the main difficulties in software engineering is that the requirements of the customer are prone to change while software is being developed. Spiraling, or evolutionary, development processes try to face this problem by building the software system *incrementally*, and by interacting with the customer after completion of each increment. Thus, requirements are not fixed at the beginning of the development, but instead converge during several cycles of increments and feedback phases.

Common incremental process models include Boehm's spiral (meta) model (Boehm, 1988), the clean-room reference model (CRM; Prowell et al., 1999), Extreme Programming (XP; Beck, 1999), and rapid and evolutionary prototyping approaches.

The CRM relies on box models at each iteration of the development process. In each cycle, requirements are specified in a black box view that associates stimulus histories (inputs) with responses (outputs). In the state box view, these history/response relations are implemented by state machines; finally, the clear box view consists of actual code. All documents are subject to thorough reviews; requirements are traced through the documents. Software tests rely on statistical usage models (which obviously raises the question of how to determine these models). In general, in the CRM tests are regarded not only as a means of detecting errors in the system, but also as a means of measuring the success of the process.

Extreme programming takes a less formal approach to incremental development. Apart from management issues or the pair programming principle, XP is based on two main concepts: Firstly, artifacts of the process are written as executable code in an ordinary, usually object-oriented, programming language. Secondly, XP makes heavy use of testing. Programmers write test cases to ensure the plausibility of their code; they—preferably together with the customer—also write functional test cases to ensure that the code satisfies the customer's demands. Both classes of tests are supposed

to be written before the code itself, and they are written in the same programming language.

An intuitive objection to incremental processes is that it is not always obvious in which direction development should proceed: As Michael Jackson puts it in the context of top-down development, the problem here also is that “you must already have solved the problem before the solution process is begun” (Jackson, 1995). Otherwise, the system will become cluttered, difficult to understand, and to communicate to the customers.

One solution might be to use standard reference architectures to guide development. In any case, the system description might need to be cleaned up after each increment; this approach is referred to as refactoring (Fowler, 1999). Refactoring is one of the foundations of XP. In fact, XP denies the absolute need for an a priori architectural design and demands that the architecture evolves along with the product.

Neither CRM nor XP are formal in the mathematical sense. While the box model of CRM and its associated concept of refinement are said to be based on the mathematical notion of referential transparency, this is not reflected in the description techniques. The state machines of the state boxes have no formal semantics on which a refinement or equivalence concept could be based.

XP, on the other hand, is based on common programming languages, and explicitly disavows any connection to mathematical principles. Nevertheless, refactoring, which is essential in XP, requires at least an intuitive concept of equivalence of programs (Philipps and Rumpe, 2001).

It is interesting that both CRM and XP make use of more than one kind of description of a system: Before executable code is written, CRM requires the specification of the system functionality as history/response pairs. In XP, system functionality is captured by functional tests. These functional tests can be regarded as specifications of the system to be designed while plausibility tests are intended to check for programmers' errors such as forgotten cases, range errors, and possibly deficiencies, contradictions, or omissions in the functional tests.

2.2. Model-based development

Incremental processes are not tied to programming languages. Modeling languages fit the demands of an incremental process in many ways better than programming languages: Their higher level of abstraction can lead to higher productivity of the programmers; their suggestive notations ease interaction with the customer and other programmers (the difficulty of understanding pure code is one of the reasons behind the XP principles of pair programming and collective code ownership; on the other hand, sometimes code is easier

to understand than any other specification, if it is devoid of the typical clutter of complete programs). This is especially important for embedded systems, where hardware and software components have to be engineered simultaneously. Choosing the right abstractions for a modeling language clearly is of paramount importance when it comes to actually specifying systems and analyzing the specifications.

For embedded systems various forms of state transition diagrams, like Statecharts, SDL or the diagrams of ROOM or UML-RT are commonly used. Sticking to the graphical specification formalisms of the CASE tool *AutoFocus* (Huber et al., 1996) exhibits, however, some advantages, since they can be given a precise and—a point that is often neglected—a *simple* formal semantics. Precision and simplicity of the semantics are a prerequisite for the operational understanding of the models, for simulation and code generation, for the definition of an equivalence concept that can be the basis of refactoring patterns, for the integration with formal verification and validation tools (Philipps and Slotosch, 1999), and—as exposed later in the paper—for the automatic generation of test cases. The formalization of behavior models is transparent: Apart from a basic operational understanding, customers are not expected to be confronted with the semantics.

With code generators, models can be translated to executable programs in languages such as C, Java, ADA or Prolog; thus, early lock-in to a certain programming language can be avoided. On the other hand, there is no reason to be dogmatic: Modeling languages can be integrated with hand-written code where convenient; models can serve as skeletons where custom code is filled in; sometimes, parts of the code for the final implementation will have to be rewritten for performance or memory space reasons. In these cases, however, the requirements will be clear, and this task seems to be easier than to use code from the beginning. Earlier work contains a more comprehensive discussion of model based development (Schätz et al., 2002).

2.3. Model-based tests

Code generation is not the only application of models. With the frugal semantics of *AutoFocus*, it is possible to automatically extract test cases from the model with explicit test case specifications in the form of sequence or interaction diagrams, or implicitly from some structural coverage criterion. The first kind of tests corresponds to functional specifications or to the tests of XP, used for interaction with the customer; the second kind is useful for plausibility checks that help to discover inconsistencies or missing cases in the model.

Fig. 1 demonstrates the use of model-based tests in incremental development. The model is incrementally developed towards a final implementation. As the increments are based on customer interaction and (possibly changing) requirements, some development steps may need to be undone.

For each increment, test cases are derived; conversely, test cases may influence the current model, as errors or ambiguities in the model are discovered by testing.

Test cases from earlier design increments can be used as test cases for later ones: One goal is to use earlier test cases as regression tests throughout the development. More importantly, earlier test specifications have an impact on the development of new tests: Since existing model components restrict the possible inputs of new components, existing tests can be used (a) as test case specifications for the newly composed component after omitting inputs or outputs, or (b) to narrow the solution space for new test case specifications. New components can restrict the inputs to existing components, and consequently, existing test cases may be modified: They might be redundant, as they cover behavior that is not feasible any more, or they can be optimized to take advantage of the reduced solution space (Bender et al., 2002).

The development steps result in a final implementation that is either generated automatically, or written by hand in case the generated code fails to satisfy performance

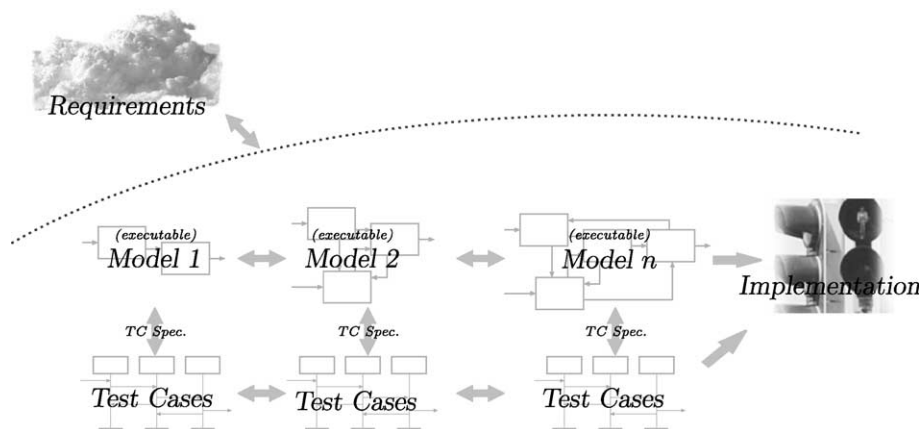


Fig. 1. Modeling and testing in incremental design processes.

or memory requirements. The final set of test cases is used to test the implementation itself: For generated code, the need arises because of the necessity of testing the software together with existing hardware in the system; for hand-written code, they are, in addition, used to test conformance with the final model (cf. Section 6). Scenarios of model-based testing are discussed by Pretschner and Philipps (2002).

The whole process is influenced by a changing set of requirements; conversely, customer feedback from models and test results can lead to a modification of requirements. Requirements can also directly influence the implementation, for example nonfunctional requirements such as coding style guidelines.

3. Behavior models

AUTOFOCUS¹ is a tool for developing graphical specifications of embedded systems based on a simple, formally defined semantics. It supports four different views on the system model to describe different system aspects: structure, behavior, interaction, and data and function definitions.

3.1. Description techniques

In AUTOFOCUS, a distributed system is a network of components that communicate via directed channels. Similar to the UML-RT, component networks are specified in *system structure diagrams* (SSDs). An example for an SSD is shown in the top half of Fig. 3. Rectangles represent components, and arrows between them represent channels. Channels are typed, and they are connected to components via ports. SSDs can be hierarchically refined: Each component may itself contain a subnetwork. Ports of components in the top level network are meant to be connected to the outside world; they form the system's interface to its environment.

The *behavior* of a component is described by a *state transition diagram* (STD). The bottom half of Fig. 3 shows two typical STDs. Initial states are marked with a black dot. An STD consists of a set of control states, transitions and local variables. The local variables build up the component's data state space.

A transition consists of several parts: A label, a guard, input statements, output statements and an action list (Fig. 4). The guard is a boolean expression that refers to local variables and to pattern variables which are bound by the input statements. Input statements consist of a port name followed by a question mark and a pattern. Analogously, output statements consist of a port name and an expression separated by an exclamation-

mark. Actions are assignments to local data state variables; they refer to local and pattern variables.

In addition to SSDs and STDs, AUTOFOCUS provides *message sequence charts* (MSCs); they are used to describe the interaction between components, for example for the visualization of simulations, and for the specification of test cases (Wimmel et al., 2000). Fig. 6 shows an example MSC.

For the specification of user defined, possibly recursive, data types and functions, AUTOFOCUS uses data type definitions (DTDs). The definitions in DTDs are written in a language quite similar to the functional programming language Haskell (Thompson, 1999), but with an eager evaluation semantics. Fig. 5 shows a part of a DTD.

3.2. The traffic light example

As a running example, a traffic light system from earlier work is considered (Philipps and Schmidt, 1997). The traffic light system is shown in Fig. 2. It controls the signals of a two-road intersection; each road has two lanes. There are no special provisions for pedestrians or for left or right turns.

Each lane has a signal lamp and an inductive loop to detect waiting or approaching cars. Both the signal lamps and the inductive loops can fail. Upon failure of a signal lamp, the controller enters a special mode where the yellow lights on the secondary road are blinking. Failure of the loops cannot be detected by the controller. Instead, a loop is assumed to be defect when it continuously signals a car for 10 min.

The traffic light controller can operate in four modes:

Daytime mode: In this mode, a road where a waiting car is detected by an induction loop should enter its green phase after at most 4 min. Additionally, every green phase lasts at least 45 s.

Nighttime mode: In nighttime mode, each road is signaled "Red" by default. An approaching car causes

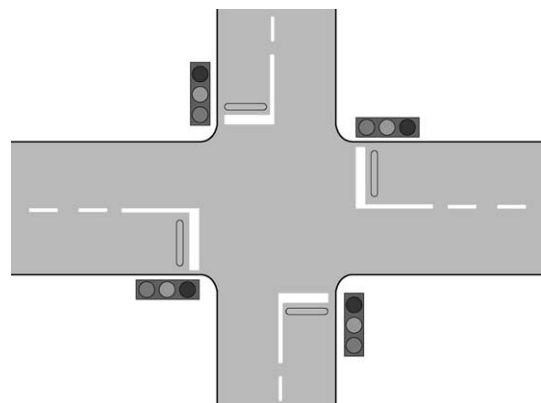


Fig. 2. Traffic intersection.

¹ <http://autofocus.in.tum.de>

the controller to enter the green phase for that direction for 10 s. With every car that is detected during this green phase in the same direction the 10 s start over. The overall duration of a green phase must not exceed 4 min.

Constant-time mode: In this mode, each road alternately enters its green phase for 3 min. Constant-time mode is entered automatically when an inductive loop is assumed to be defect.

Blinking mode: Here, all signal lamps on the primary road are switched off, and the signal lamps on the secondary road blink with a frequency of 0.5 Hz. Blinking mode is entered when either a lamp is defect. When the mode is entered, it must be ensured that all traffic lights switch to “Red” for 2 s before blinking begins.

The mode of the traffic controller is determined by an external mode switch. The switch can be either in daytime, nighttime, or constant time position. No further assumptions about fairness or traffic optimization are made.

The AutoFocus model. Fig. 3 shows the system structure and the state transition diagrams for two of the traffic light controller’s components. The system is structured into four components:

LoopController: This controller counts the time that each loop sensor has been active, and projects the sensor data into the *LoopData* data type (see Fig. 5 for the data type definitions). The behavior is shown at the bottom left of Fig. 3; Fig. 4 shows the definition of one of the transitions. The component accesses and modifies two local variables of type integer, *hCounter* and *vCounter*, which hold the number of seconds that the sensors on the horizontal and vertical roads, respectively, have been active.

MainControl: This component is refined into two subcomponents, which determine the controller mode (day, night, constant-time) and phase (horizontal road in green phase, vertical road in green phase, both roads in red phase), respectively. Their state transition diagrams are omitted here.

LampCheck: This is a trivial latch component that reads a status signal from each lamp and notifies the lamp controller if any bulb is damaged.

LampControl: This component controls the signal lamps based on input from the phase controller. The two roads are treated completely symmetrically. In addition, from all states except the transient states between two phases there are transitions that switch the traffic light into blinking mode upon a lamp failure.

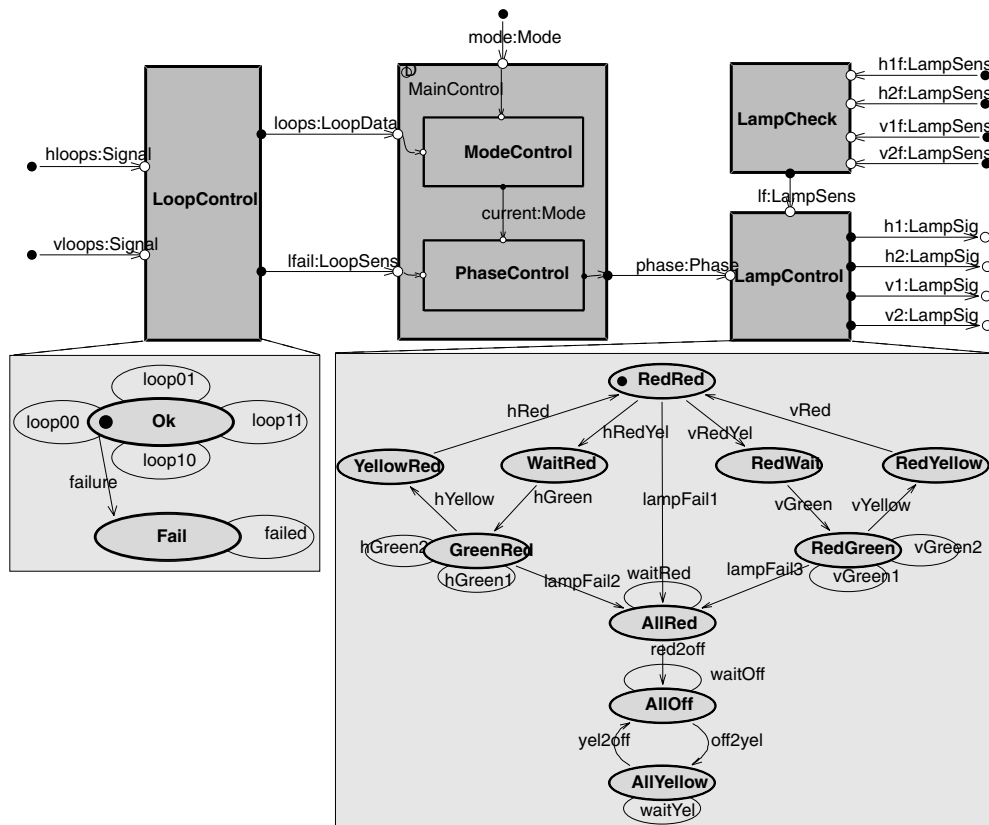


Fig. 3. Traffic lights example: structure and parts of the behavior.

Guard:	$hCounter < durLoopFail \wedge vCounter < durLoopFail$
Input:	$hloops?; vloops?Present$
Output:	$loops!Loops(False, True)$
Action:	$hCounter = 0; vCounter = vCounter + 1$

Fig. 4. Transition “loop10”.

Fig. 5 shows part of the associated data type and function definitions (the DTD) of the traffic controller.

As an aside, function definitions and pattern matching can be used to “overlay” different behaviors in a single state transition diagram. The bottom of Fig. 5 shows two function definitions to demonstrate such overlays; they are used in the guard of the phase controller to determine when to switch phases. The function `doH2V` defines the condition under which the controller switches from a horizontal road green phase to a vertical mode green phase. It has six parameters: The current mode (an enumeration of day, night and constant-time mode), two Boolean values which hold the current loop sensor status), two integers which hold the number of seconds the horizontal or vertical road, respectively, has been “idle” (i.e., since the last time a car crossed the road), and finally an integer that holds the number of seconds the current (i.e., horizontal) phase has been active. The first line of the function definition defines the condition in constant-time mode: sensor readings and the value of the idle time counters are ignored, and the function returns true only if 180 (`durConst`) seconds have passed. The second line defines the condition in day-time mode: A car must wait at the vertical road (i.e., its sensor is “True”), but the horizontal green phase

must have been active for at least 45 (`durDay`) seconds. In night-time mode, the horizontal phase must have been active for more than 4 min (240 s, `durNight`). According to the fourth line, in all other cases the transition is disabled.

The transition from the horizontal green phase to the all-red phase can only be taken in night-time mode; it is only enabled if no car passed the crossing on the horizontal road for 10 (`durTimeout`) seconds.

Similar functions are defined for the other transitions in the phase controller; transition actions increment and reset the idle counters and the phase counter.

Obviously, such a modal behavior can also be defined by hierarchical state machines; `AutoFocus` STDs can be hierarchical. The approach with a mode variable and function definitions with pattern matching is more concise, however, and leads to less redundancy in the graphical descriptions.

3.3. Operational model

A transition can fire if its guard holds and the patterns on the input statements match the values read from the component’s input channels. After execution of the transition the values in the output statements are

```

data LoopData = Loops(hLoop:Bool, vLoop:Bool);
data LoopSens = LoopFail;
data Mode = Day | Night | Const;

const durTimeout = 10;      const durNight = 240;
const durDay = 45;          const durConst = 180;
const durLoopFail = 600;

doH2V : Mode->(Bool->(Bool->(Int->(Int->(Int->Bool)))));
fun doH2V(Const,hl,vl,hi,vi,t) = (t >= durConst)
  | doH2V(Day,hl,True,hi,vi,t) = (t >= durDay)
  | doH2V(Night,hl,vl,hi,vi,t) = (t >= durNight)
  | doH2V(m,hl,vl,hi,vi,t) = False;

doH2R : Mode->(Bool->(Bool->(Int->(Int->(Int->Bool)))));
fun doH2R(Night,hl,vl,hi,vi,t) = (hi >= durTimeout)
  | doH2R(m,hl,vl,hi,vi,t) = False;

```

Fig. 5. Traffic lights example: partial data and function definition.

copied to the appropriate ports, and local variables are set according to the assignments.

AUTOFOCUS components are timed by a common global clock, i.e., they all perform their computations simultaneously. Each clock cycle consists of two steps: First each component reads the values on its input ports and computes new values for local variables and output ports. Then, the new values are copied to the output ports where they can be accessed immediately via the input ports of connected components. This cyclic operation results in a *time-synchronous* communication scheme with buffer size 1.

The choice for this synchronous scheme was motivated by the use of AUTOFOCUS and its associated validation techniques for small embedded systems: These systems are conceptually close to clocked hardware, and have low-level interfaces to sensors and actuators.

Somewhat to the authors' surprise, the scheme works rather well for some other domains, notably for smart card applications (Pretschner et al., 2001b). For yet other domains, the execution model might have to be adapted; while this causes changes in code generators, translators to verification tools and to the test case generation described in the next section, the graphical descriptions need not be adapted.

4. Model based testing

Software testing is the process of executing a piece of software in order to detect incorrect behavior (a software failure). Software failures can either originate from wrong implementations or invalid or incomplete specifications.

By using executable models in evolutionary prototyping, the testing process can start much earlier and can therefore be more effective. Model based testing covers test case derivation from models, running tests by executing the model, and running tests by executing the actual implementation. Different scenarios of model based testing are discussed by Pretschner and Philipps (2002).

4.1. Test case generation

Test case generation starts with defining a *test purpose*. In general, a test purpose is an informal formulation of the properties which, later on, will be verified by the generated test cases. From *informal* test purposes, a *formal* test case specification is derived. Both test purposes and test case specification can be either functional (i.e., require test cases to test certain functionalities without regarding the model) or structural (e.g., require test cases to obtain a certain coverage of the implemented code). A number of actual test cases is then

computed for each test case specification. Simple examples for test cases include input histories, transition tours, traces, or constraints over them. Test cases are required to be consistent with their test case specification, but they need not be executable (Lötzbeyer and Pretschner, 2000b).

To perform real tests, test cases must necessarily be executable and predict the results of the computation. Such test cases are called *test sequences*. A test sequence is a fully instantiated I/O trace that can directly be fed into an implementation or an executable system model, and after the test is performed, a *verdict* (pass, fail, inconclusive) can be assigned.

In addition to using counterexamples from model checkers—or bounded model checkers based on propositional solvers (Wimmel et al., 2000)—that exhibit the disadvantage of no control over the generated trace, AUTOFOCUS allows the generation of test cases and test sequences with CLP.

The CLP based test sequence generation overcomes these limitations. Roughly, test sequence generation proceeds as follows. The AUTOFOCUS model is translated into CLP code; so is the test case specification provided by a sequence diagram, an automaton, a temporal formula or a set of constraints. These two are conjoined with additional environmental and efficiency constraints, and the resulting program is executed, thus simulating one or more system runs. In general, the state space still is too large, and abstractions to the end of test case generation need to be manually inserted. As an example, equivalence classes of inputs may be directly encoded in the model. The resulting test cases—i.e., representations of sets of test sequences—are then instantiated into actual test sequences. Note that unlike other approaches, the presented approach does not aim at extracting a test suite that is able to determine equivalence (or implication) between a specification and an implementation; *completeness* of test cases is thus not the prime goal.

Test case generation for models that exhibit intended nondeterminism is not problematic since it is covered by Prolog's backtracking facilities. Actual implementations should be deterministic; generating test cases and verdicts for the implementation, or final model, respectively, is thus not problematic, either. AUTOFOCUS is equipped with a plug-in that checks these (final) models for inadvertent nondeterminism.

4.2. Translation into CLP

The translation of AUTOFOCUS models into CLP code is straightforward and fully automatic. For each atomic component a step predicate is introduced. This predicate represents one single step of the component. Each transition of the corresponding automaton is represented by a single rule of the step predicate.

This also includes the *idle transitions*—transitions that fire if no other can—which are not explicitly visible in the automaton. The rules have the following form:

```
stepComponent(State,          transition,
Inputs, Outputs, NextState):-
    inputCond, guard, assignments.
```

where *transition* is the transition's label, *State* and *NextState* are tuples of variables representing the system's control and data states before and after the transition, and *Inputs*, and *Outputs* are variables or tuples of variables representing channel valuations. *inputCond* directly translates the transition's matching conditions on input channels into Prolog (unification). *guard* and *assignments* are predicates which may refer to all variables. These predicates may involve constraints.

For each composed component consisting of a network of communicating subcomponents, a special scheduler rule is created that drives the subcomponents and transfers messages between subcomponents and the environment. The scheduler rule has the same signature as the step predicates of the atomic components. Thus, from a black box view, no difference is made between an atomic and a composed component, and the encoding in Prolog rules reflects the component hierarchy of the AutoFocus model.

For the top level component, an additional rule is needed that successively calls the step rule and collects the histories of states, local variables, inputs, outputs, and transitions. The number of steps is limited in order to avoid infinite runs. Details of the concrete translation are discussed by Lötzbeyer and Pretschner (2000a,b).

In addition to the Prolog rules which model the transitions of the system, constraint solvers for the evaluation of the predicates (functions, data types) are needed. For integer or real typed variables, constraint solvers for finite or real domain variables are available, and for the functional data types, a corresponding constraint solver is generated automatically by using constraint handling rules (CHR; Frühwirth, 1998). The advantages of the CLP encoding are manifold: Infinite data can be handled by using appropriate constraint solvers, and a flexible search is performed by Prolog's backtracking and instantiation abilities. Beyond that, the translation preserves the structure of the model. In the authors' opinion, this is an important point, as it is a prerequisite to adjust the search for special purposes (e.g. reach certain states or fire a specific transition, see below). In addition to a quite abstract encoding, the use of constraints entitles one to a priori prune the search tree (e.g., testing properties that consist of implications), and to handle negative properties by delaying their instantiations.

4.3. Test case generation as a search problem

Assume that a test case specification is given as a simple existential statement: "eventually, state *s* is to be reached". Here *s* is a (partially) instantiated vector of the system's state space, involving control as well as data states. Concatenating several (atomic) existential test case specifications allows the definition of complex scenarios—functional tests—as well as structural testing criteria which boil down to such concatenations. With suitable transformations, this is true even for coverage criteria based on control flow. Since state spaces tend to be large, determining a trace that leads to *s* amounts to finding the needle in a haystack. Clearly, simply enumerating all states (and the traces that lead to them) is a solution only for very small state spaces. Note, however, that even without the following necessary optimizations, the above mentioned generation procedure enumerates *sets* of states rather than individual states: the example of $l(t) > 5$ for the value of a local variable at a particular moment in time is easily extended to a guard specifying that input *i* at time *t* is *cmd*₁, *cmd*₂, or *cmd*₃ which is translated into the constraint $i(t) \in \bigcup_{j=1}^3 \{cmd_j\}$. Rather than three enumerated traces, one is sufficient. This constraint may be strengthened in subsequent computation steps.

4.4. The role of constraints

Constraints are used for several purposes. It has been demonstrated above that they enable one to compute with sets of traces. Reconsider the problem of coping with inequality. A guard specified by $l(t) \neq 5$ is unfolded by a model checker like SPIN into every single possible number other than five. Additionally, sets of states are stored by means of constrained formulas. Checking whether or not a state has already been visited is then achieved by checking entailment rather than by checking syntactic equality (modulo renaming). Furthermore, with constraints it is possible to compactly store states by means of generalized BDDs on domains other than the Booleans (with the usual problems like finding good variable orderings which is not the subject of this paper).

While not visiting states twice this is not always desired when designing test cases—often, systems suffer from strange behaviors of the kind "*n* times everything is fine, the *n* + 1st time, it is not"—it often is a key prerequisite to finding test cases at all.

Furthermore, constraints are a user-friendly way of directly specifying test cases (e.g., "reach *q*, and signal *s* occurs at most twice", or "*s*₁ has to be output before *s*₂"). For this purpose, several temporal operators have been predefined with CHR. They may also be used for defining environment assumptions or efficiency constraints (experience has shown that with the presented system and on-line statistics, test engineers often are able

to see why a test case cannot be found which enables them to directly exclude certain parts of the search space by forbidding certain transitions or input values). In fact, this possibility of interactively searching test cases is the key to graceful degradation of the presented approach. Note that while CHRs are a comfortable specification language, these constraints can also be described by automata, regular expressions, sequence diagrams, and so on. Also note that when computing with sets of traces, the corresponding constraints (test cases) have to be instantiated in order to yield test sequences. This is done either at random, by solving constraints, and, in the presence of ordered types, by using boundary analyses or techniques from equivalence testing.

4.5. Search strategies

Several search strategies have been implemented in the tool prototype (Pretschner, 2001). Breadth first search is prohibitive for all but very small systems. The remainder of this paragraph will thus be concerned with several variants of depth first search for existential properties. The crucial point obviously is to find good transition orderings for every component within each step of the composed system—while backtracking ensures completeness, it usually is impossible to explore the whole state graph which is the root of the desire of exploring “interesting” parts of the search space at the beginning of the searching procedure.

An interleaving choice of transitions ensures that within every component, if a control state is visited a second time, then the outgoing transition is chosen fairly from all possible ones: if the fanout is n , then this strategy tries to ensure that for n visits of the same control state, n different outgoing transitions are chosen. Obviously, this goal is not always achievable since the other components or the environment may prevent certain transitions from being fired. The idea, however, is to “spread” the search as much as possible.

In a competitive concurrent setting—several search “threads” execute simultaneously and have access to the *same* set of hitherto visited states—it is not surprising that a random choice of transitions yields a good performance. An explicit annotation of transitions with probabilities is also possible (cf. the approach of environment profiles taken in the CRM); note that due to the interdependence of components, these local transition probabilities may in practice not always be achievable.

Thus far, the search strategies are not specialized to a given test case specification. Best-first or A* strategies provide a means of doing so (Pretschner, 2001). The idea here is to compute, for each (control and data) state, a distance measure to the system state that is specified by the test case specification. Before choosing a transition

from a given (control) state, all possible transitions are executed, and the distance of their destination states to the desired state is computed. Transitions are then ordered according to the “fitness” of the respective destination states. Obviously, this approach only works if a corresponding “fitness function” can be determined easily—a task that is particularly difficult if the system’s state space is not “continuous” w.r.t. that fitness function (in which case minimal random deviations from the “optimal” ordering are used, as, for instance, done in simulated annealing). In the following, a simple but powerful heuristics (fitness function) is described that is based on the structure of one automaton and that allows for *statically* encoding it into the model: the fitness does not have to be computed at runtime.

Roughly, the idea is to compute from each control state c the shortest paths to a transition or control state of interest, d , where each of these paths starts with a different transition that emanates from c . In its simplest version, the length of a path is measured by the number of transitions it contains. The transition ordering for c is then w.r.t. the length of the shortest paths that lead to d . As we show below, this simple heuristics can be extended with information on which transitions in the automaton lead the system state to a state that is “closer” to d . In this case, fitness may have to be determined at run time; in addition to the number of transitions to a particular control states, examples for components of the fitness include the distance between a local variable’s current and desired value, the number of satisfied conjuncts that form a (control and data) state that is to be reached, etc. An example for more general fitness functions is given in Section 4.6.

These heuristics are rather local in that they usually take into account just one component of interest. Current work focuses on how to extend such heuristics to more than one component. A related problem is that of how to order *components* in the *step* formulas—a first heuristics is to put components with best-first-strategies at the beginning and the other ones, with, for instance, random transition choice, at the end. This puts “more” emphasis on the component with a more sophisticated strategy; the simple construction ensures that the “sophisticated” component restricts the other ones rather than the other way around. Further ideas include heuristics that are based on the number of constraints, the number of variables they involve, and the cardinality of their domains.

On a finer granularity, the ordering in which variables of a particular transition are bound to their respective domains, also turns out to be crucial for good computational performance. There is experimental evidence that maximizing the number of choicepoints (a simple approximating estimate is the product of number of input channels with the cardinality of their type) at the beginning and putting components (or transitions,

respectively) that are likely to generate less choicepoints at the end, yields better performance. The reason, of course, is the nature of depth-first search. However, a detailed analysis of this kind of heuristics is not the subject of this paper.

4.6. The traffic light example

Two functional test cases for the traffic light example are considered. The first consists of finding a trace that drives the system into the constant mode when one or both loops are defect (test case specification: `ch_mem(LoopFail, lfail)`), and the second consists of driving it into the blinking mode (i.e., a bulb is defect; specification: `st_mem(All0, LampControl)`).² Remember that component `LoopControl` is associated with two variables that count the number of loop failures for each lane, `vcounter` and `hcounter`. One would like the loops to be defect for 10 consecutive minutes and then enter state σ_{Fail} which directly yields the fitness function for component `LoopControl`:

$$\varphi_1(t) = 1/(600 - \text{hcounter} + 600 - \text{vcounter} + \delta(\text{dest}(t), \sigma_{\text{Fail}})) \quad (1)$$

where $\text{dest}(t)$ is the target control state of transition t , and δ is the smallest number of transitions between the two argument control states. The fitness function for the other test case takes into account nothing but the shortest paths between control states in component `LampControl`:

$$\varphi_2(t) = 1/(\delta(\text{dest}(t), \sigma_{\text{AllYellow}})) \quad (2)$$

since the blinking mode is entered when this control state is reached. This second fitness function can be hard coded into the model which is not directly clear for φ_1 . It is, however, the case, since transition `loop00` in component `LoopControl` takes care of the case where both loops are defect. $\varphi'_1(\text{loop00}) = 2$, $\varphi'_1(\text{loop01}) = \varphi'_1(\text{loop10}) = 1$, and $\varphi'_1(t) = 0$ for all other transitions t enables one to achieve the same effects with a statical analysis. Note, however, that (1) this simple strategy is not always possible, and (2) that the test case specification could also consist of finding a trace that exhibits a defect bulb (transition `lampFail1` in component `LampControl` eventually fires).

The computed test sequence for the second test case specification is displayed in Fig. 6; the other one is omitted for the sake of brevity, as its length exceeds 600 steps. The tool prototype computes all sequences within fractions of a second. The model checkers connected to `AutoFocus` have not been able to compute these se-

quences. No statistics are provided here, since their significance is unclear without comparative numbers—for some performance measures, see previous work (Pretschner, 2001; Pretschner et al., 2001a).

5. Related work

The general schema of generating test cases is similar to Howden's proposition (Howden, 1975). The intricate question of what should be tested (Goodenough and Gerhart, 1975; Weyuker, 1986) remains unsolved. Zhu et al. (1997) review the use of structural coverage criteria as test case specifications in their overview article. Ntafos (1988) compares several control flow coverage criteria; Frankl and Weyuker (1988) do so for data flow testing criteria. Vilkomir and Bowen (2001) provide a formalization of control flow coverage criteria that can serve for the automatic generation not of test cases but rather test case specifications. Finally, random testing as test case specification turns out to be powerful when input data partitioning—and consequently structural coverage—is not based on the likelihood of errors (Duran and Ntafos, 1984; Hamlet and Taylor, 1990).

The following contains an overview over the state of the art in test case generation techniques for reactive systems. Test patterns and test case generation for object oriented systems are deliberately not taken into account (Binder, 2001).

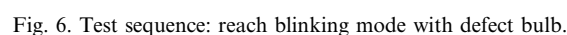
Synchronous languages. Much of the literature on test case generation for reactive systems is related to the French school of synchronous programming languages. This is no coincidence: Their precise semantics and operational model lend themselves well to algorithmic analysis.

The Lurette tool (Raymond et al., 1998) aims at black-box generating test sequences for Lustre programs. The idea is to encode the notion of “relevant” behaviors (or test sequences) in an observer node—basically, a set of constraints over Booleans and real variables which take into account the directly preceding outputs of the program under test. These are used for the computation of new inputs to the program: random values that satisfy all constraints.

Lutess (du Bousquet et al., 2000) follows the same basic ideas for black-box testing invariance properties. I/O data types are restricted to the Booleans; the “observer” (or test case specification, respectively) is encoded as a BDD. At each step, a value satisfying the constraints given by this BDD is chosen. Lutess provides several strategies for property-based, random, stochastic (environment) profile-based, and behavioral pattern-based choice of inputs.

GATeL (Marre and Arnould, 2000) computes test sequences for Lustre programs by solving systems of constraints. These constraint systems directly reflect the

² Test case specifications expressed as constraints can contain possibly constrained variables as well as predefined temporal subformulas over state, transition, local variables, and communication histories.



Structured data types as well as recursive functions on the “transition” level are not supported.

LOTOS/SDL/Z/EFSMs. The TGV tool (Fernandez et al., 1996) computes test sequences for LOTOS and

SDL specifications translated into input/output labeled transition systems (IOLTS). Essentially, the synchronous product of a test purpose and the system under test is traversed in order to decorate this product with verdicts and timers. The result is a test graph (representing test sequences and their respective verdicts). Data states in the IOLTS are not explicitly taken into account.

Like TGV, some of the test case generators connected to TorX (Vries et al., 2000) build on the so-called ioco testing theory (Tretmans, 1996) for labeled transition systems. TorX is a test tool architecture that allows black-box batch-wise and on-the-fly testing of SDL, LOTOS, and Promela specifications. The generation algorithm for on-the-fly testing is similar to that of Lurette or Lutess in that they are generated along with corresponding verdicts while traversing a product of system (I/O) and test case. Rusu et al. (2000) explicitly extend IOLTS with variables. The basic ideas of the underlying test case generator are similar to that of TGV (product of test case specification and system under test). The motivation for this is not to explicitly encode a system's data state in an LTS which severely suffers from the state space explosion problem.

The approaches based on IOLTS as an intermediate language require the explicit computation of the specification's product automaton (specifications tend to consist of more than one process or, in the presented context, component) as well as this product automaton's product with the test case specification. At least in the case of IOSTS, this leads to a large number of unsatisfiable transition guards which, during state space exploration, have to be computed for each product transition. Rusu et al. (2000) use PVS and HyTech for the expensive automatic computation of invariants that may enable one to statically exclude these transitions. Current work concentrates on also including such static analysis procedures; note, however, that in the presented compositional approach, the above mentioned failing guards need to be computed only once.

AutoLink (Koch et al., 1998) is a tool for generation of test cases from SDL specifications. It works on LTS rather than extended finite state machines. No symbolic execution for reducing the state space is performed.

Bourhfir et al. (1996) present different test generation techniques for EFSM based systems. Among others, Sadeghipour (1998) and Burton et al. (2001) use the Isabelle theorem prover for generating test cases from Z specifications with extended finite state machines or StateCharts.

Model checking-based approaches. The obvious idea of directly using counterexamples from (symbolic as well as explicit-state) model checkers as test cases has been considered widely; see the work of Ammann et al. (1998) or, for the use of bounded model checking, that of Wimmel et al. (2000).

While not directly aiming at testing, Edelkamp et al. (2001) augment SPIN's search strategies with A* search. Fitness functions are based on a rather syntactic degree of partial satisfaction of formulas and on the euclidian distance between states (in the context of evolutionary testing and with a slightly different motivation, this approach is also taken by Tracey (2000)). The structure of the never-claim is taken into account for increasing efficiency when model checking liveness properties. Conceptually, this approach is quite similar to the presented efforts in using best-first heuristics; the difference lies in the use of constraints for symbolically executing several traces simultaneously and for storing large sets of states as well as on a methodological level: it is not stated exactly which properties are to be checked (i.e., how to derive a set of finite test sequences for, e.g., an invariance property).

The Java PathFinder project (Visser et al., 2000) deploys explicit model checking technology for analyzing Java programs with similar heuristics (Groce and Visser, 2002); due to the infinite state space of such programs, this is an approach similar to ours (differing in the technology as well as the emphasis on existential properties).

Constraint and logic based approaches, symbolic execution. The approach of Ciarlini and Frühwirth (1999) is similar (symbolic execution on the grounds of CLP), differing from the presented work w.r.t. compositionality, the degree of automation as well as the combination of state machines with specifications in a functional language for guards and postconditions, i.e., the input language.

Fribourg (1999) notes the resemblance of constraint solving in Logic Programming and model checking of (infinite-state) systems. Delzanno and Podelski (1999) identify implication in LP with transitions and, with a set of sound widening abstractions, model check several infinite-state systems in this way. Cui et al. (1998) use tabled resolution (Warren, 1992) in XSB Prolog for the implementation of an efficient model checker.

Symbolic execution for test case generation has been discussed for a long time. Examples include the work of King (1976), Clarke (1976), Howden (1977, 1978), and Ramamoorthy et al. (1976). Because of then not readily available performant constraint solvers, these systems often needed help in terms of, for instance, prespecified paths.

Meudec (2000) uses CLP for symbolic execution of SPARK (SPADE Ada Kernel) code in order to compute test cases. The focus is neither on reactive nor on concurrent systems. In fact, SPARK is used for high integrity software and deliberately does not support tasks. Concurrency has to be implemented explicitly by writing appropriate schedulers.

Legard and Peureux (2001) use a constraint solver to compute test cases from B specifications. While similar

to the presented approach, there are some technical differences. Constraints are not used for storing hitherto visited states. Directed search does not seem to be implemented. The authors' experience is that these techniques are necessary to get an acceptable performance, even for small systems.

Logic variables naturally lend themselves to be used in symbolic execution and test case generation. Consequently, there are many Prolog-based approaches. Denney (1991), for instance, abstracts recursion in Prolog programs and heuristically defines equivalence classes on the resulting abstract flow graph that are then used as a basis for symbolic execution. Since Prolog is not typed, all these approaches do not take into account types. Types, however, are taken into account in the QuickCheck tool for (functional) Haskell programs (Claessen and Hughes, 2000). The basic idea is to give a separate specification of a program and randomly generate test cases. *AUTOFOCUS* makes use of an eager functional language; testing functional programs is thus one part of the presented approach.

6. Conclusion

This paper describes the state of the authors' continuing efforts in specification based test sequence generation and its embedding in an incremental development process for reactive systems software.

In addition to checking interaction with hardware devices, the lack of verified compilers and code generators still necessitates testing the implementation: checking the model clearly is not sufficient (Fetzer, 1988).

In order to assess the scalability of the CLP based approach, the authors are carrying out an industrial size case study with a large German manufacturer of smart cards. Results of a feasibility study (Pretschner et al., 2001b)—where both incremental models and the actual hardware were tested—have given reason to be optimistic but show that more intelligence in the process of search is necessary; further experiments based on the approaches outlined in Section 4 are definitely needed.

Conceptually, the presented approach is quite similar to (bounded) explicit-state on-the-fly model checking as performed by tools like SPIN (Holzmann, 1997), where test cases, environmental and efficiency constraints are implemented by temporal logic formulas, or by “never claims” given by a Büchi automaton. At the technical level, the approach in this paper is dedicated to existential properties that describe functional test cases or coverage criteria while the power of model checkers is usually exploited to show universal properties that hold for all possible system runs. It is thus possible to make use of the large body of work on graph search algorithms and heuristics to optimize test case generation. Moreover, the class of systems that can be handled is

not restricted to finite ones: For example, recursive data types or real numbers are handled in exactly the same way as finite enumeration types.

On the other hand, existential properties are not a very convincing argument for testing, say, invariance properties. Nevertheless, test engineers try to find “meaningful” test cases for such properties, based on typical heuristics like boundary analysis. While with the presented approach it is easy to randomly compute large sets of test cases, a methodological transition from universal to existential properties based on such heuristics seems to be necessary, and is the subject of current work. The idea is to negate the state part of an invariance formula. By using directed search as described above, it is then attempted to reach such a state that violates the original invariance property. If such a state is reached, the invariance property does not hold. If it is not, one might at least have come “close” to a violating state. By analogy with traditional boundary testing, this might turn out to yield meaningful test cases.

The presented approach aims at testing models as well as implementations. Using the same model for both production code and test case generation is problematic since in this case, the model's role as an oracle (does the implementation's output correspond to the expected—i.e., the model's—output?) has to be questioned. If the actual implementation is not generated but hand-coded, the model may well serve as an oracle. With the ever-increasing cost of active quality assurance for safety critical or widely deployed systems (e.g., smart cards), this approach of using “validation models” is, nonetheless, likely to pay off. Conversely, if production code is generated, outputs of generated test cases then have to be manually checked for correctness which is still likely to be advantageous when compared to manually computing all test sequences (Pretschner and Philipps, 2002).

Whether (1) code and test cases are generated from models, or (2) models for test case generation are possibly automatically built on the grounds of existing code, or (3) models and code are developed simultaneously, is independent of the problem of relating different levels of abstractions. The very nature of a model is that it is a simplification. However, it is not always obvious how to concretize test cases in order to get executable inputs for an actual implementation. Whether or not this obstacle can be overcome depends on the application domain. This is a general problem of model-based development: Generated code can only be as precise and deterministic as the original models. In the realm of mixed discrete-continuous embedded systems, common tools like ASCET-SD (ETAS) or production code generators for Matlab (Mathworks)—like Beacon (ADI) or TargetLink (dSpace)—show, somewhat surprisingly, that the problem is not desperately difficult. For business information systems, there in general is no

need to cope with the intricacies of system-level interaction with sensors and actuators.

When seen as a search problem, the obvious hindrance to successful test case generation is the size of the state space. Abstraction techniques and sophisticated symbolic execution technology can alleviate this problem. However, in the context of modular testing, the complexity of computation and communication semantics has a direct influence on the success of the algorithm. A simple semantics—like the simple clock-synchronous semantics of *AUTOFOCUS* as opposed to, for instance, the rather complex semantics advocated by some Statecharts dialects (von der Beeck, 1994)—not only is a prerequisite for effective use of verification technology but also for the intellectual mastery of a system (Leveson et al., 1999). Whether or not the presented technology carries over to more powerful and thus complicated specification formalisms in a scalable manner remains to be assessed.

Finally, experience with industrial partners shows that a customized management systems for testing—especially for regression tests—is at least as important as a systematic generation of test cases, while being cheaper and easier to adopt. Without such a system, advanced test case generation techniques are not likely to be adopted by practitioners.

References

- Ambler, S., 2002. *Agile Modeling*. J. Wiley and Sons.
- Ammann, P., Black, P., Majurski, W., 1998. Using model checking to generate tests from specifications. In: *Proceedings of the 2nd IEEE International Conference on Formal Engineering Methods*. pp. 46–54.
- Beck, K., 1999. *Extreme Programming Explained: Embrace Change*. Addison Wesley.
- Bender, K., Broy, M., Péter, I., Pretschner, A., Stauner, T., 2002. Model based development of hybrid systems: specification, simulation, test case generation. In: *Modelling, Analysis and Design of Hybrid Systems*. LNCS, vol. 279. Springer, pp. 37–52.
- Binder, R., 2001. *Testing Object-Oriented Systems: Models, Patterns, and Tools*. Addison-Wesley.
- Boehm, B., 1988. A spiral model of software development and enhancement. *Computer* 21 (2), 61–72.
- Boger, M., Baier, T., Wienberg, F., Lamersdorf, W., 2000. Extreme modeling. In: *Proceedings of the Extreme Programming and Flexible Processes in SW Engineering*.
- Bourhfir, C., Dssouli, R., Aboulhamid, E., 1996. Automatic test generation for EFSM-based systems. Tech. Rep. IRO 1043, University of Montreal.
- Brooks, F., 1986. No Silver Bullet. In: *Proceedings of the 10th IFIP World Computing Conference*. pp. 1069–1076.
- Burton, S., Clark, J., McDermid, J., 2001. Automatic generation of tests from Statechart specifications. In: *Proceedings of the Formal Approaches to Testing of Software*. pp. 31–46.
- Ciarlini, A., Frühwirth, T., 1999. Using constraint logic programming for software validation. In: *5th Workshop on the German–Brazilian Bilateral Programme for Scientific and Technological Cooperation*.
- Claessen, K., Hughes, J., 2000. QuickCheck: A lightweight tool for random testing of Haskell programs. In: *Proceedings of the International Conference on Functional Programming*.
- Clarke, L., 1976. A system to generate test data and symbolically execute programs. *IEEE Transactions on Software Engineering* SE-2 (3), 215–222.
- Cui, B., Dong, Y., Du, X., Narayan Kumar, K., Ramakrishnan, C., Ramakrishnan, I., Roychoudhury, A., Smolka, S., Warren, D., 1998. *Logic programming and model checking*. Springer LNCS 1490. pp. 1–20.
- Delzanno, G., Podelski, A., 1999. Model checking in CLP. In: *Proceedings of the Tools and Algorithms for Construction and Analysis of Systems*. pp. 223–239.
- Denney, R., 1991. Test-case generation from prolog-based specifications. *IEEE Software* 8 (2), 49–57.
- du Bousquet, L., Ouabdesselam, F., Parisis, I., Richier, J.-L., Zuanon, N., 2000. Specification-based testing of synchronous software. In: *Proceedings of the 5th International Workshop on Formal Methods for Industrial Critical Systems*.
- Duran, J., Ntafos, S., 1984. An evaluation of random testing. *IEEE Transactions on Software Engineering* SE-10 (4), 438–444.
- Edelkamp, S., Lluch-Lafuente, A., Leue, S., 2001. Directed explicit model checking with HSF-SPIN. In: *8th International SPIN Workshop on Model Checking Software*.
- Fernandez, J.-C., Jard, C., Jéron, T., Viho, C., July 1996. Using on-the-fly verification techniques for the generation of test suites. In: *Proceedings of the 8th International Conference on Computer-Aided Verification*.
- Fetzer, J., 1988. Program verification: The very idea. *CACM* 37 (9), 1048–1063.
- Fowler, M., 1999. *Refactoring-Improving the Design of Existing Code*. Addison Wesley.
- Frankl, P., Weyuker, E., 1988. An applicable family of data flow testing criteria. *IEEE Transactions on Software Engineering* 14 (10), 1483–1498.
- Fribourg, L., 1999. Constraint logic programming applied to model checking. In: *Proceedings of the 9th International Workshop on Logic-based Program Synthesis and Transformation (LOPSTR'99)*. LNCS 1817. Springer-Verlag, Venice.
- Frühwirth, T., 1998. Theory and practice of constraint handling rules. *Journal of Logic Programming* 37 (1–3), 95–138.
- Goodenough, J., Gerhart, S., 1975. Toward a theory of test data selection. *IEEE Transactions on Software Engineering* SE-1 (2), 156–173.
- Groce, A., Visser, W., 2002. Model Checking Java Programs using Structural Heuristics. In: *Proceedings of the International Symposium on Software Testing and Analysis*.
- Hamlet, D., Taylor, R., 1990. Partition test does not inspire confidence. *IEEE Transactions on Software Engineering* 16 (12), 1402–1411.
- Holzmann, G.J., 1997. The spin model checker. *IEEE Transactions on Software Engineering* 23 (5), 279–295.
- Howden, W., 1975. Methodology for the generation of program test data. *IEEE Transactions on Computers* C-24 (5), 554–560.
- Howden, W., 1977. Symbolic testing and the DISSECT symbolic evaluation system. *IEEE Transactions on Software Engineering* SE-3 (4), 266–278.
- Howden, W., 1978. An evaluation of the effectiveness of symbolic testing. *Software—Practice and Experience* 8, 381–397.
- Huber, F., Schätz, B., Schmidt, A., Spies, K., 1996. Autofocus—a tool for distributed systems specification. In: *FTRTFT'96*, LNCS 1135.
- Jackson, M., 1995. *Software Requirements and Specifications*. Addison-Wesley.
- King, J., 1976. Symbolic execution and program testing. *CACM* 19 (7), 385–394.
- Koch, B., Grabowski, J., Hogrefe, D., Schmitt, M., 1998. AutoLink—a tool for automatic test generation from SDL specifications. In:

- Proceedings of the IEEE International Workshop on Industrial Strength Formal Specification Techniques.
- Legeard, B., Peureux, F., June 2001. Génération de séquences de tests à partir d'une spécification B en PLC ensembliste. In: *Proceedings of the Approches Formelles dans l'Assistance au Développement de Logiciels*. pp. 113–130.
- Leveson, N.G., Heimdahl, M., Reese, J.D., 1999. Designing specification languages for process control systems: Lessons learned and steps to the future. In: *FOSE '99 (Foundations of Software Engineering)*.
- Lötzbeyer, H., Pretschner, A., 2000a. AutoFocus on Constraint Logic Programming. In: *Proceedings of the (Constraint) Logic Programming and Software Engineering*.
- Lötzbeyer, H., Pretschner, A., 2000b. Testing Concurrent Reactive Systems with Constraint Logic Programming. In: *Proceedings of the 2nd Workshop on Rule-Based Constraint Reasoning and Programming*.
- Marre, B., Arnould, A., 2000. Test Sequence Generation from Lustre Descriptions: GATEL. In: *Proceedings of the 15th IEEE International Conf on Automated Software Engineering (ASE'00)*. Grenoble.
- Meudec, C., 2000. ATGen: automatic test data generation using constraint logic programming and symbolic execution. In: *Proceedings of the 1st International Workshop on Automated Program Analysis, Testing, and Verification*. Limerick.
- Ntafos, S., 1988. A comparison of some structural testing strategies. *IEEE Transactions on Software Engineering* 14 (6), 868–874.
- Philipps, J., Rumpe, B., October 2001. Roots of refactoring. In: *Proceedings of the 10th OOPSLA Workshop on Behavioral Semantics: Back to Basics*. pp. 187–199.
- Philipps, J., Schmidt, A., 1997. Traffic flow by data flow. Tech. Rep. TUM-19718, Institut für Informatik, Technische Universität München.
- Philipps, J., Slotosch, O., 1999. The quest for correct systems: Model checking of diagrams and datatypes. In: *APSEC'99*. IEEE Computer Society, pp. 449–458.
- Pretschner, A., 2001. Classical search strategies for test case generation with Constraint Logic Programming. In: *Proceedings of the Formal Approaches to Testing of Software*. pp. 47–60.
- Pretschner, A., Lötzbeyer, H., Philipps, J., 2001a. Model based testing in evolutionary software development. In: *Proceedings of the 11th IEEE International Workshop on Rapid System Prototyping*. pp. 155–160.
- Pretschner, A., Slotosch, O., Lötzbeyer, H., Aiglstorfer, E., Kriebel, S., 2001b. Model based testing for real: The inhouse card case study. In: *Proceedings of the 6th International Workshop on Formal Methods for Industrial Critical Systems*. pp. 79–94.
- Pretschner, A., Philipps, J., 2002. Szenarien modellbasierten Testens. Tech. Rep. TUM-I0205, Institut für Informatik, Technische Universität München.
- Prowell, S., Trammell, C., Linger, R., Poore, J., 1999. *Cleanroom Software Engineering*. Addison Wesley.
- Ramamoorthy, C., Ho, S., Chen, W., 1976. On the automated generation of program test data. *IEEE Transactions on Software Engineering SE-2* (4), 293–300.
- Raymond, P., Weber, D., Nicollin, X., Halbwachs, N., 1998. Automatic testing of reactive systems. In: *Proceedings of the 19th IEEE Real-Time Systems Symposium*.
- Rusu, V., du Bousquet, L., Jéron, T., 2000. An Approach to Symbolic Test Generation. In: *Proceedings of the Integrated Formal Methods*.
- Sadeghipour, S., 1998. Testing cyclic software components of reactive systems on the basis of formal specifications. Ph.D. Thesis, TU Berlin.
- Schätz, B., Pretschner, A., Huber, F., Philipps, J., 2002. Model-based development. Tech. Rep. TUM-I0204, Institut für Informatik, Technische Universität München.
- Thompson, S., 1999. *Haskell: The Craft of Functional Programming*. Addison-Wesley.
- Tracey, N., 2000. A search-based automated test-data generation framework for safety-critical software. Ph.D. Thesis, University of York.
- Tretmans, J., 1996. Test generation with inputs, outputs and repetitive quiescence. *Software—Concepts and Tools* 17 (3), 103–120.
- Vilkomir, S., Bowen, J., 2001. Formalization of control-flow criteria of software testing. Tech. Rep. SBU-CISM-01-01, South Bank University.
- Visser, W., Havelund, K., Brat, G., Park, S., 2000. Java PathFinder—Second Generation of a Java Model Checker. In: *Proceedings of the Workshop on Advances in Verification*.
- von der Beeck, M., 1994. A comparison of statecharts variants. In: Langmaack, H., de Roever, W.-P., Vytupil, J. (Eds.), *Proceedings of the Formal Techniques in Real-Time and Fault-Tolerant Systems (FTRTFT'94)*. Lecture Notes in Computer Science, vol. 863. Springer, pp. 128–148.
- Vries, R.d., Tretmans, J., Belinfante, A., Feenstra, J., Feijs, L., Mauw, S., Goga, N., Heerink, L., Heer, A.d., 2000. Côte de Resyste in Progress. In: *Progress 2000—Workshop on Embedded Systems*. pp. 141–148.
- Warren, D., 1992. Memoing for logic programs. *Communications of the ACM* 35 (3), 93–111.
- Weyuker, E., 1986. Axiomatizing software test data adequacy. *IEEE Transactions on Software Engineering SE-12* (12), 1128–1138.
- Wimmel, G., Lötzbeyer, H., Pretschner, A., Slotosch, O., 2000. Specification based test sequence generation with propositional logic. *Journal of Software Testing, Validation, and Reliability* 10 (4), 229–248.
- Zhu, H., Hall, P., May, J., 1997. Software unit test coverage and adequacy. *ACM Computing Surveys* 29 (4), 366–427.